

Parallel and Distributed Computing

Assignment 4

Submitted to: Dr. Saif Nalband

Course: UCS645

Exercise 1: Ring Communication

Problem:

Write an MPI program where each process sends a message to the next process in a ring topology. Process 0 sends to Process 1, Process 1 to Process 2, and so on, with the last process sending back to Process 0.

Requirements:

- Initialize value as **100** in Process 0
- Each process adds its rank before forwarding
- Display value at each step
- Final value should return to Process 0

Hint:

Use modulo operation:

```
next = (rank + 1) % size
```

Execution:

```
mpicc -o ring_comm ring_comm.c
```

```
mpirun --allow-run-as-root --oversubscribe -np 4 ./ring_comm
```

Sample Output:

Process 0 starts with 100
Process 1 received 100
Process 1 updated value: 101
Process 2 received 101
Process 2 updated value: 103
Process 3 received 103
Process 3 updated value: 106
Process 0 received final value: 106

Exercise 2: Parallel Array Sum

Problem:

Develop an MPI program to compute the sum of an array in parallel by dividing work among processes.

Execution:

```
mpicc -o array_sum array_sum.c  
mpirun --allow-run-as-root --oversubscribe -np 4 ./array_sum
```

Output:

Total sum: 5050 (expected 5050)

Exercise 3: Maximum and Minimum using MPI

Problem:

Each process generates random numbers and determines local min/max. Compute global min and max across all processes.

Requirements:

- Generate 10 random values per process (0–1000)
- Compute local min and max
- Use MPI_Reduce for global values
- Display which process produced them

Execution:

```
mpicc -o max_min max_min.c
```

```
mpirun --allow-run-as-root --oversubscribe -np 4 ./max_min
```

Output:

Global Maximum: 992 (Process 1)

Global Minimum: 41 (Process 3)

Exercise 4: Parallel Dot Product**Problem:**

Calculate the dot product of two vectors using parallel processing.

Given:

- $A = [1, 2, 3, 4, 5, 6, 7, 8]$
- $B = [8, 7, 6, 5, 4, 3, 2, 1]$

Approach:

- Distribute vectors using MPI_Scatter
- Compute partial products
- Use MPI_Reduce to get final result

Execution:

```
mpicc -o dot_product dot_product.c
```

```
mpirun --allow-run-as-root --oversubscribe -np 4 ./dot_product
```

Output:

Dot product: 120 (expected 120)

Conclusion

All programs were successfully executed using MPICH on a parallel environment.